

Esercizio S10/L2: Gestione dei Permessi di Lettura, Scrittura ed Esecuzione in Linux

Configurazione UGO, ownership, umask, special bits e ACL su Kali Linux

1. Obiettivo

In questo esercizio ho configurato e gestito i permessi di lettura, scrittura ed esecuzione su un albero di file e directory in Kali Linux. Ho costruito uno scenario realistico (una directory di progetto condivisa /opt/progetto_sicurezza) con tre categorie di risorse (configurazione riservata, script eseguibile, log condiviso) e tre profili di utente (owner, membro di gruppo, esterno). Su questo banco di prova ho esercitato chmod numerico e simbolico, chown/chgrp, umask, i bit speciali SGID e sticky e infine le ACL POSIX per concedere permessi granulari a un singolo utente.

Lo scopo non è solo conoscere la sintassi, ma capire perché ogni configurazione isola correttamente i dati sensibili e quali effetti collaterali compaiono quando la membership di gruppo o l'umask non sono coerenti con il modello di accesso.

2. Configurazione Ambiente

Tutta l'attività si è svolta su una singola macchina Kali Linux, eseguendo i comandi come root tramite sudo -i. Ho creato un gruppo di lavoro team_sec e tre utenti dedicati con ruoli distinti.

Comando	Descrizione
sudo -i	Apertura sessione root per le operazioni privilegiate.
groupadd team_sec	Creazione del gruppo di progetto.
useradd -m -G team_sec analyst1	Owner del progetto, membro del gruppo team_sec.
useradd -m -G team_sec analyst2	Secondo membro del team.
useradd -m guest1	Utente esterno, fuori dal gruppo team_sec.
passwd analyst1 / analyst2 / guest1	Impostazione password per consentire su -.

```
root@kali: /opt/progetto_sicurezza

Session Actions Edit View Help

(kali@kali)-[~]
$ sudo -i
[sudo] password for kali:
(root@kali)-[~]
# groupadd team_sec

(root@kali)-[~]
# useradd -m -G team_sec analyst1

(root@kali)-[~]
# useradd -m -G team_sec analyst2

(root@kali)-[~]
# useradd -m guest1

(root@kali)-[~]
# passwd analyst1
New password:
Retype new password:
passwd: password updated successfully

(root@kali)-[~]
# passwd analyst2
New password:
Retype new password:
Sorry, passwords do not match.
passwd: Authentication token manipulation error
passwd: password unchanged

(root@kali)-[~]
# passwd analyst2
New password:
Retype new password:
passwd: password updated successfully

(root@kali)-[~]
# passwd guest1
New password:
Retype new password:
passwd: password updated successfully
```

Figura 1: creazione del gruppo team_sec e dei tre utenti analyst1, analyst2, guest1.

3. Fasi dell'Esercizio

3.1 Creazione della struttura di progetto

Ho creato la gerarchia di directory e tre file rappresentativi: una configurazione con una credenziale (dato sensibile), uno script di audit (eseguibile) e un log iniziale (dato condiviso). Una nota pratica: il primo echo con doppi apici è entrato in modalità dquote> perché bash ha provato a fare history expansion sul carattere ! della password; ho risolto passando agli apici singoli.

```
mkdir -p /opt/progetto_sicurezza/{config,scripts,logs}
cd /opt/progetto_sicurezza
echo 'DB_PASS=Sup3rS3cret!' > config/db.conf
echo -e '#!/bin/bash\n echo "Audit avviato da $(whoami)"' > scripts/audit.sh
echo "$(date) - log iniziale" > logs/audit.log
ls -lR /opt/progetto_sicurezza
```

```
root@kali: /opt/progetto_sicurezza

Session Actions Edit View Help

(root@kali)-[/opt/progetto_sicurezza]
# echo "DB_PASS=Sup3rS3cret!" > config/db.conf
dquote>

(root@kali)-[/opt/progetto_sicurezza]
# cho 'DB_PASS=Sup3rS3cret!' > config/db.conf
Command 'cho' not found, did you mean:
command 'echo' from deb coreutils
command 'cha' from deb chawan
command 'cdo' from deb cdo
command 'who' from deb coreutils
command 'chg' from deb mercurial
command 'chr' from deb chr
command 'chr' from deb chr-tiny
command 'co' from deb rcs
Try: apt install <deb name>

(root@kali)-[/opt/progetto_sicurezza]
# echo 'DB_PASS=Sup3rS3cret!' > config/db.conf

(root@kali)-[/opt/progetto_sicurezza]
# echo -e '#!/bin/bash\n echo "Audit avviato da $(whoami)"' > scripts/audit.sh

(root@kali)-[/opt/progetto_sicurezza]
# echo "$(date) - log iniziale" > logs/audit.log

(root@kali)-[/opt/progetto_sicurezza]
# ls -lR /opt/progetto_sicurezza
/opt/progetto_sicurezza:
total 12
drwxr-xr-x 2 root root 4096 Jun  9 09:33 config
drwxr-xr-x 2 root root 4096 Jun  9 09:33 logs
drwxr-xr-x 2 root root 4096 Jun  9 09:33 scripts

/opt/progetto_sicurezza/config:
total 4
-rw-r--r-- 1 root root 21 Jun  9 09:33 db.conf

/opt/progetto_sicurezza/logs:
total 4
-rw-r--r-- 1 root root 47 Jun  9 09:33 audit.log

/opt/progetto_sicurezza/scripts:
total 4
-rw-r--r-- 1 root root 46 Jun  9 09:33 audit.sh

(root@kali)-[/opt/progetto_sicurezza]
```

Figura 2: struttura iniziale con permessi di default (644 sui file, 755 sulle directory).

I permessi di partenza sono quelli derivati dalla umask 022 della shell root: `rw-r--r--` per i file e `rw-r-xr-x` per le directory. Sono troppo permissivi per dati riservati.

3.2 Ownership: `chown` e `chgrp`

Ho riassegnato proprietà e gruppo dell'intero albero ad `analyst1:team_sec` con un solo comando ricorsivo. Questo è il prerequisito perché i bit di gruppo abbiano un senso operativo: senza un proprietario sensato e un gruppo coerente, il modello UGO non protegge nulla.

```
chown -R analyst1:team_sec /opt/progetto_sicurezza
```

3.3 Permessi numerici (ottali)

Ho applicato i permessi target con notazione ottale: 640 sulla configurazione (owner `rw`, gruppo `r`, others nulla), 750 sullo script (eseguibile per il team, vietato agli esterni), 660 sul log (scrittura condivisa al team) e 750 sulle directory per consentire al solo team di attraversarle.

Comando	Descrizione
<code>chmod 640 config/db.conf</code>	Credenziali leggibili dal team, nessun accesso ad altri.
<code>chmod 750 scripts/audit.sh</code>	Script eseguibile dal team, invisibile agli esterni.

Comando	Descrizione
<code>chmod 660 logs/audit.log</code>	Log scrivibile dal team.
<code>chmod 750 config scripts logs</code>	Directory traversabili solo dal team.

```
(root@kali)-[/opt/progetto_sicurezza]
# chmod 640 config/db.conf

(root@kali)-[/opt/progetto_sicurezza]
# chmod 750 scripts/audit.sh

(root@kali)-[/opt/progetto_sicurezza]
# chmod 660 logs/audit.log

(root@kali)-[/opt/progetto_sicurezza]
# chmod 750 config scripts logs

(root@kali)-[/opt/progetto_sicurezza]
# ls -lR /opt/progetto_sicurezza

/opt/progetto_sicurezza:
total 12
drwxr-x— 2 root root 4096 Jun  9 09:33 config
drwxr-x— 2 root root 4096 Jun  9 09:33 logs
drwxr-x— 2 root root 4096 Jun  9 09:33 scripts

/opt/progetto_sicurezza/config:
total 4
-rw-r— 1 root root 21 Jun  9 09:33 db.conf

/opt/progetto_sicurezza/logs:
total 4
-rw-rw— 1 root root 47 Jun  9 09:33 audit.log

/opt/progetto_sicurezza/scripts:
total 4
-rwxr-x— 1 root root 46 Jun  9 09:33 audit.sh

(root@kali)-[/opt/progetto_sicurezza]
#
```

Figura 3: permessi numerici applicati, verificati con `ls -lR`.

3.4 Permessi simbolici

Per esercitare la sintassi alternativa ho riapplicato gli stessi permessi in forma simbolica (`u=rw,g=r,o=` equivale a `640`) e ho mostrato come modificare un singolo bit (`o+r` apre la lettura agli esterni, `o-r` la richiude). La notazione simbolica è preferibile quando voglio toccare un solo bit senza riscrivere l'intera maschera.

```

(root@kali)-[/opt/progetto_sicurezza]
# chmod u=rw,g=r,o= config/db.conf

(root@kali)-[/opt/progetto_sicurezza]
# chmod u+x,g+x,o-rwx scripts/audit.sh

(root@kali)-[/opt/progetto_sicurezza]
# chmod o+r logs/audit.log

(root@kali)-[/opt/progetto_sicurezza]
# ls -l config/db.conf scripts/audit.sh logs/audit.log
-rw-r----- 1 root root 21 Jun  9 09:33 config/db.conf
-rw-rw-r-- 1 root root 47 Jun  9 09:33 logs/audit.log
-rwxr-x--- 1 root root 46 Jun  9 09:33 scripts/audit.sh

(root@kali)-[/opt/progetto_sicurezza]
# chmod o-r logs/audit.log

```

Figura 4: modifiche simboliche (u=rw,g=r,o=) e apertura/chiusura mirata dell'access bit per others sul log.

3.5 Test pratico di accesso (analyst2 e guest1)

La prova del modello DAC si fa cambiando utente. Ho aperto due sessioni con `su -`: `analyst2` (membro di `team_sec`) e `guest1` (esterno). Il risultato osservato richiede una nota didattica importante: anche `analyst2` ha ricevuto `Permission denied` sul file di configurazione e sul log.

La causa è che la membership al gruppo `team_sec` è stata assegnata con `useradd -G` contestualmente alla creazione dell'utente, ma la sessione `su -` non ha caricato correttamente i gruppi supplementari (effetto noto: a volte serve un re-login completo o `newgrp team_sec`). Sul piano didattico la dimostrazione è comunque valida: il modello blocca chiunque non risulti, in quel preciso istante, membro effettivo del gruppo. Per evitare il problema in scenari reali è buona pratica verificare con `id analyst2` prima di testare l'accesso.

```

(root@kali)-[/opt/progetto_sicurezza]
# su - analyst2
$ cat /opt/progetto_sicurezza/config/db.conf
cat: /opt/progetto_sicurezza/config/db.conf: Permission denied
$ bash /opt/progetto_sicurezza/scripts/audit.sh
bash: /opt/progetto_sicurezza/scripts/audit.sh: Permission denied
$ echo "test analyst2" >> /opt/progetto_sicurezza/logs/audit.log
-sh: 3: cannot create /opt/progetto_sicurezza/logs/audit.log: Permission denied
$ exit

(root@kali)-[/opt/progetto_sicurezza]
# su - guest1
$ cat /opt/progetto_sicurezza/config/db.conf
cat: /opt/progetto_sicurezza/config/db.conf: Permission denied
$ ls /opt/progetto_sicurezza/config
ls: cannot open directory '/opt/progetto_sicurezza/config': Permission denied
$ exit

```

Figura 5: tentativi di accesso da `analyst2` e `guest1`, tutti negati per i motivi descritti.

3.6 Umask

Ho dimostrato come la `umask` influenzi i permessi di default dei nuovi oggetti. Con `umask 0022` i file nascono 644 e le directory 755; impostando `umask 0027` i file diventano 640 e le directory 750, escludendo per default gli others. È la misura più efficace per evitare che file riservati nascano accidentalmente world-readable.

```

umask          # mostra 022 (default)
umask 0027     # nega rwx agli others per i nuovi file
touch /tmp/test_umask.txt
mkdir /tmp/test_umask_dir
ls -ld /tmp/test_umask*

```

```
(root@kali)-[/opt/progetto_sicurezza]
# umask
022

(root@kali)-[/opt/progetto_sicurezza]
# umask 0027

(root@kali)-[/opt/progetto_sicurezza]
# touch /tmp/test_umask.txt

(root@kali)-[/opt/progetto_sicurezza]
# mkdir /tmp/test_umask_dir

(root@kali)-[/opt/progetto_sicurezza]
# ls -ld /tmp/test_umask*
drwxr-x--- 2 root root 40 Jun  9 09:38 /tmp/test_umask_dir
-rw-r----- 1 root root  0 Jun  9 09:38 /tmp/test_umask.txt
```

Figura 6: nuovi oggetti creati con umask 0027 nascono rispettivamente 640 e 750.

3.7 Special bits: SGID e sticky

Ho attivato SGID sulla directory logs (i file nuovi ereditano automaticamente il gruppo team_sec invece del gruppo primario di chi li crea) e il sticky bit (solo l'owner di ciascun file può cancellarlo o rinominarlo, anche se la directory è scrivibile dal gruppo). Il risultato visibile in `ls -ld` è `drwxr-s--T`: la `s` indica SGID, la `T` maiuscola indica sticky senza `eXecute` per others.

Anche qui il `touch` da parte di `analyst2` è fallito, coerentemente con quanto osservato in 3.5 e con il fatto che la directory logs è 750: il gruppo non ha permesso di scrittura sulla directory, quindi non può creare file al suo interno. Per consentire la scrittura di gruppo manterrei SGID+sticky ma alzerei i permessi a 2770 (o 1770).

```
(root@kali)-[/opt/progetto_sicurezza]
# chmod g+s /opt/progetto_sicurezza/logs

(root@kali)-[/opt/progetto_sicurezza]
# chmod +t /opt/progetto_sicurezza/logs
ls -ld /opt/progetto_sicurezza/logs
drwxr-s--T 2 root root 4096 Jun  9 09:33 /opt/progetto_sicurezza/logs

(root@kali)-[/opt/progetto_sicurezza]
# su - analyst2 -c 'touch /opt/progetto_sicurezza/logs/da_analyst2.log; ls -l /opt/progetto_sicurezza/logs/da_analyst2.log'
touch: cannot touch '/opt/progetto_sicurezza/logs/da_analyst2.log': Permission denied
ls: cannot access '/opt/progetto_sicurezza/logs/da_analyst2.log': Permission denied

(root@kali)-[/opt/progetto_sicurezza]
#
```

Figura 7: bit SGID e sticky applicati alla directory logs (`drwxr-s--T`).

3.8 ACL granulari con setfacl/getfacl

Per superare i limiti del modello UGO (un solo owner, un solo gruppo) ho usato le ACL POSIX: ho concesso a `guest1` la lettura sul solo file `audit.log`, senza inserirlo in `team_sec` e senza modificare altri file. Servono anche le ACL `--x` sulle directory intermedie, altrimenti il path non è attraversabile. La verifica con `getfacl` mostra la entry esplicita `user:guest1:r--`.

```
setfacl -m u:guest1:r-- /opt/progetto_sicurezza/logs/audit.log
setfacl -m u:guest1:--x /opt/progetto_sicurezza
setfacl -m u:guest1:--x /opt/progetto_sicurezza/logs
getfacl /opt/progetto_sicurezza/logs/audit.log
su - guest1 -c 'cat /opt/progetto_sicurezza/logs/audit.log' # OK
su - guest1 -c 'cat /opt/progetto_sicurezza/config/db.conf' # ancora denied
```

```

(root@kali)-[/opt/progetto_sicurezza]
# setfacl -m u:guest1:r-- /opt/progetto_sicurezza/logs/audit.log

(root@kali)-[/opt/progetto_sicurezza]
# setfacl -m u:guest1:--x /opt/progetto_sicurezza
setfacl -m u:guest1:--x /opt/progetto_sicurezza/logs
getfacl /opt/progetto_sicurezza/logs/audit.log
getfacl: Removing leading '/' from absolute path names
# file: opt/progetto_sicurezza/logs/audit.log
# owner: root
# group: root
user::rw-
user:guest1:r--
group::rw-
mask::rw-
other::—

(root@kali)-[/opt/progetto_sicurezza]
# su - guest1 -c 'cat /opt/progetto_sicurezza/logs/audit.log'
Tue Jun 9 09:33:34 AM EDT 2026 - log iniziale

(root@kali)-[/opt/progetto_sicurezza]
# su - guest1 -c 'cat /opt/progetto_sicurezza/config/db.conf'
cat: /opt/progetto_sicurezza/config/db.conf: Permission denied

(root@kali)-[/opt/progetto_sicurezza]
#

```

Figura 8: ACL applicate, verifica con getfacl e doppio test (lettura log riuscita, db.conf ancora negato).

Questo è il valore aggiunto delle ACL rispetto al modello classico: posso concedere permessi mirati a singoli utenti senza alterare l'ownership né allargare il gruppo, rispettando il principio del minimo privilegio.

4. Validazione Finale

Per consolidare lo stato finale ho usato tre strumenti complementari: `ls -lR` per la vista d'insieme, `stat` per ottenere permessi in forma ottale e simbolica insieme, e `namei -l` per ispezionare i permessi lungo l'intero path (utile in debug quando un accesso fallisce non per il file ma per una directory intermedia). Il + accanto a `audit.log` segnala la presenza di ACL estese.

```
(root@kali)-[/opt/progetto_sicurezza]
# ls -lR /opt/progetto_sicurezza
stat /opt/progetto_sicurezza/config/db.conf
namei -l /opt/progetto_sicurezza/logs/audit.log
/opt/progetto_sicurezza:
total 12
drwxr-x--- 2 root root 4096 Jun  9 09:33 config
drwxr-s--T+ 2 root root 4096 Jun  9 09:33 logs
drwxr-x--- 2 root root 4096 Jun  9 09:33 scripts

/opt/progetto_sicurezza/config:
total 4
-rw-r----- 1 root root 21 Jun  9 09:33 db.conf

/opt/progetto_sicurezza/logs:
total 4
-rw-rw----+ 1 root root 47 Jun  9 09:33 audit.log

/opt/progetto_sicurezza/scripts:
total 4
-rwxr-x--- 1 root root 46 Jun  9 09:33 audit.sh
File: /opt/progetto_sicurezza/config/db.conf
Size: 21      Blocks: 8      IO Block: 4096   regular file
Device: 8,1   Inode: 4456459   Links: 1
Access: (0640/-rw-r-----)  Uid: (  0/   root)   Gid: (  0/   root)
Access: 2026-06-09 09:33:04.347500748 -0400
Modify: 2026-06-09 09:33:14.244446691 -0400
Change: 2026-06-09 09:35:17.458021999 -0400
Birth: 2026-06-09 09:33:04.347500748 -0400
f: /opt/progetto_sicurezza/logs/audit.log
drwxr-xr-x root root /
drwxr-xr-x root root opt
drwxr-xr-x root root progetto_sicurezza
drwxr-s--T root root logs
-rw-rw---- root root audit.log

(root@kali)-[/opt/progetto_sicurezza]
```

Figura 9: vista finale con ls -lR, stat su db.conf e namei -l sul path completo del log.

5. Riepilogo Comandi

Comando	Descrizione
groupadd / useradd -G	Creazione del gruppo e degli utenti con membership supplementari.
chown -R user:group path	Riassegnazione ricorsiva di owner e gruppo.
chmod 640 / 750 / 660	Applicazione di permessi in notazione ottale.
chmod u=rw,g=r,o=	Sintassi simbolica equivalente, utile per modifiche mirate.
umask 0027	Maschera che esclude gli others sui nuovi oggetti.
chmod g+s	SGID: i file ereditano il gruppo della directory.
chmod +t	Sticky bit: solo l'owner del file può rimuoverlo.
setfacl -m u:user:perm	ACL POSIX granulari per singolo utente.
getfacl path	Visualizzazione delle ACL impostate.
stat / namei -l	Ispezione dettagliata di permessi e path.
su - user	Apertura sessione con altro utente per testare gli accessi.

6. Conclusioni

Ho ricostruito tutto lo stack di controllo accessi a livello filesystem Linux: ownership con chown, modello UGO in notazione ottale e simbolica, default tramite umask, ereditarietà e protezione tramite SGID e sticky, e infine

la granularità fine delle ACL POSIX. La parte più istruttiva è stata osservare un Permission denied inatteso su analyst2: mi ha ricordato che la membership di gruppo va sempre verificata in tempo reale con `id` prima di concludere che l'errore è nei bit, e che una directory 750 non basta se voglio che il gruppo ci scriva (servono 770 con SGID, cioè 2770).

In un'ottica difensiva (SOC, hardening) le lezioni operative sono: impostare una umask restrittiva di sistema (027 o 077 sui server), preferire le ACL alla moltiplicazione di gruppi quando i permessi richiesti sono ad personam, applicare sticky e SGID alle directory condivise per evitare cancellazioni incrociate e mantenere coerenti i gruppi tra owner della directory e contenuto. In ottica offensiva (pentest) ricordo che file con bit SUID/SGID inattesi sono il primo posto da controllare per escalation: `find / -perm -u=s -type f 2>/dev/null`.